INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

WEEK6 LECTURE1

INTEL X86-64 ASSEMBLY LANGUAGE INSTRUCTIONS

Address Computation Instruction

- **`leaq` *Src*, *Dst*** ***(Load Effective Address Instruction)***
  - *Src* is address mode expression
  - Set *Dst* to address denoted by expression

%rdx	***0xf000***
%rcx	***0x0100***

```
leaq (%rdx, %rcx, 4), %rbx
%rbx = 0xf000 + 0x100 * 4 = 0xf400
```

- Uses
  - Computing addresses without a memory reference
    - E.g., translation of `p = &x[i];`
  - Computing arithmetic expressions of the form x + k*y
    - k = 1, 2, 4, or 8
- Example

```
long m12(long x)
{
  return x*12;
}
```

```
leaq (%rdi,%rdi,2), %rax # t <- x+x*2
salq $2, %rax            # return t<<2
```

Converted to ASM by compiler:

Load effective address leaq Example

- **leaq vs movq**

%rdi = 0x2000, %rsi= 4

leaq (%rdi, %rsi,4), %rax

EA = %rdi + %rsi * 4 = 0x2000 + 4*4 = 0x2000+ 0x10=0x2010

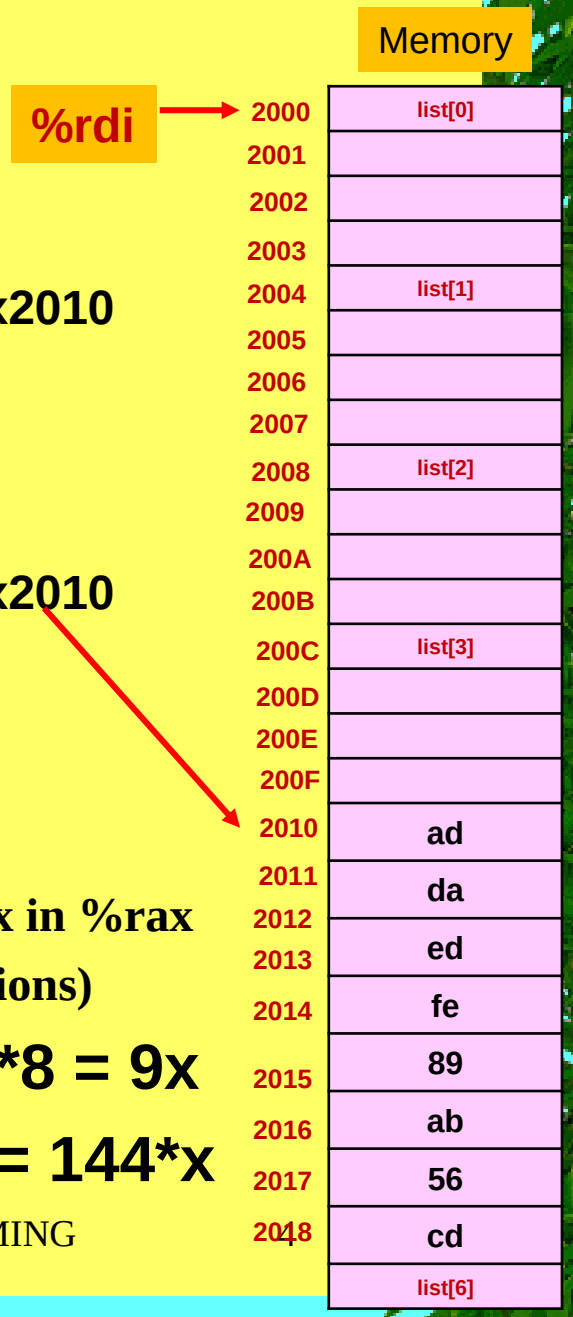
%rax = 0x2010

movq (%rdi, %rsi,4), %rax

EA = %rdi + %rsi * 4 = 0x2000 + 4*4 = 0x2000+ 0x10=0x2010

Copy 8 bytes from Mem(x2010) to %rax

%rax = 0xcd56ab89feeddaad



Q)Write x86-64 assembly language instructions to multiply x in %rax register by 144 using only leaq and shlq (shift left instructions)

leaq (%rax, %rax, 8), %rax x + x*8 = 9x

shlq $4, %rax 9x * 16 = 144*x

Arithmetic Operations

- **Two Operand Instructions:**

Format		*Computation*	
`addq`	*Src,Dest*	Dest = Dest + Src	add
`subq`	*Src,Dest*	Dest = Dest – Src	subtract
`cmpq`	*Src,Dest*	Dest – Src	compare
`imulq`	*Src,Dest*	Dest = Dest * Src	integer or signed multiply
`mulq`	*Src,Dest*	Dest = Dest * Src	unsigned multiply
`salq`	*Src,Dest*	Dest = Dest << Src	arithmetic shift left (*shlq*)
`shlq`	*Src,Dest*	Dest = Dest << Src	logical shift left
`sarq`	*Src,Dest*	Dest = Dest >> Src	*Arithmetic shift right*
`shrq`	*Src,Dest*	Dest = Dest >> Src	*Logical right*
`xorq`	*Src,Dest*	Dest = Dest ^ Src	Exclusive or
`andq`	*Src,Dest*	Dest = Dest & Src	and
`orq`	*Src,Dest*	Dest = Dest | Src	or

- Watch out for argument order!
- No distinction between signed and unsigned int (why?)

Arithmetic Operations

One Operand Instructions

`incq` *Dest*	*Dest = Dest + 1*	*Increment*
`decq` *Dest*	*Dest = Dest – 1*	*Decrement*
`negq` *Dest*	*Dest = – Dest*	*Two's complement*
`notq` *Dest*	*Dest = ~Dest*	*One's complement*

ROW MAJOR ORDER

int mat[4][4]; **No. of Rows R=4, No. of Columns C=4,Base address = mat**

……. **S=Size of matrix element = sizeof(int) = 4**

a = mat[2][3]; **row index i = 2, column index j=3**

Compute the effective address EA of the matrix element mat[2][3]

EA = Base address + No. of columns * row index * S + column index * S

= **mat + C * i * S + j * S** = mat+ 4 * 2 * 4 + 3 * 4 = mat+32+12 **= mat + 44**

REGISTER MAPPING: %rsi = mat, %rax= C=4, %rcx=i=2, %rdx=j=3

leaq (%rsi, %rdx, 4), %rbx

%rbx=%rsi+%rdx*4=mat+j*4

mul %rcx

%rax= %rax*%rcx=C*i

leaq (%rbx, %rax, 4), %rbx

%rbx=%rbx+%rax*4

%rbx = mat+j*4 + C*i*4

mat[2][3]

mat[4][4] =

	C0	C1	C2	C3
R0	23	45	-67	89
R1	-14	93	27	-94
R2	09	-47	38	**76**
R3	86	54	-83	35

Memory

2000	23
2004	45
2008	-67
20012	89
20016	-14
20020	93
2024	27
2028	-94
2032	09
2036	-47
2040	38
2044	76
2048	86
2052	54
2056	-83
2060	35

Arithmetic Expression Example

```
long arith
(long x, long y, long z)
{
  long t1 = x+y;
  long t2 = z+t1;
  long t3 = x+4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

Interesting Instructions

- `leaq`: address Computation
- `salq`: shift
- `imulq`: multiplication
  - But, only used once

%rdi <– x, %rsi<– y, %rdx<– z

%rax<– t1 z+t1 -> %rax = t2

%rdx <- %rsi + %rsi*2 = 3*y

shift left %rdx 4 times = y*48 =t4

%rcx <- 4 + %rdi + %rdx = 4+x+t4=t5

%rax <- %rax * %rcx = t2*t5

Return %rax

```
arith:
  leaq    (%rdi,%rsi), %rax
  addq    %rdx, %rax
  leaq    (%rsi,%rsi,2), %rdx
  salq    $4, %rdx
  leaq    4(%rdi,%rdx), %rcx
  imulq   %rcx, %rax
  ret
```

Understanding Arithmetic Expression Example

```
long arith
(long x, long y, long z)
{
  long t1 = x+y;
  long t2 = z+t1;
  long t3 = x+4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

```
arith:
   leaq    (%rdi,%rsi), %rax    # t1
   addq    %rdx, %rax           # t2
   leaq    (%rsi,%rsi,2), %rdx
   salq    $4, %rdx             # t4
   leaq    4(%rdi,%rdx), %rcx   # t5
   imulq   %rcx, %rax           # rval
   ret
```

Register	Use(s)
%rdi	**Argument** *x*
%rsi	**Argument** *y*
%rdx	**Argument** *z*
%rax	*t1*, *t2*, *rval*
%rdx	*t4*
%rcx	*t5*

X86-64 Assembly Language Programming

DATA MOVEMENT INSTRUCTIONS

Instruction		Effect	Description
MOV	S, D	D ← S	Move
movb		Move byte	
movw		Move word	
movl		Move double word	
movq		Move Quad Word	
MOVS	S, D	D ← SignExtend(S)	Move with sign extension
movsbw		Move sign-extended byte to word	
movsbl		Move sign-extended byte to double word	
movswl		Move sign-extended word to double word	
movswq, movslq		Move sign-extended word to quad word and longword to Quad Word	
MOVZ	S, D	D ← ZeroExtend(S)	Move with zero extension
movzbw		Move zero-extended byte to word	
movzbl		Move zero-extended byte to double word	
movzwl		Move zero-extended word to double word	
movzwq, movzlq		Move zero-extended word to quad word and longword to Quad	
pushq	S	R[%rsp] ← R[%rsp] - 8 M[R[%rsp]] ← S	Push quad word
popq	D	D ← M[R[%rsp]] R[%rsp] ← R[%rsp] + 8	Pop quad word

X86-64 Assembly Language Programming

- **Zero-extending & Sign-extending data movement instructions**
  - **These instructions have a register or memory location as the source and a register as the destination**

Instruction	operation	Description
movz src, dst	**dst <-- ZeroExtend(src)**	**Move with zero extension**
movzbw	movzbw %bl, %ax	**Move zero-extended byte to word**
movzbl	movzbl %bl, %eax	**Move zero-extended byte to double word**
movzwl	movzwl %bx, %eax	**Move zero-extended word to double word**
movzbq	movzbq %bl, %rax	**Move zero-extended byte to quad word**
movzwq movzlq	movzwq %bx, %rax movzwq %ebx, %rax	**Move zero-extended word to quad word** **Move zero-extended long word to quad word**
movs src,dst	**dst <-- SignExtend(src)**	**Move with sign extension**
movsbw	movsbw %bl, %ax	**Move sign-extended byte to word**
movsbl	movsbl %bl, %eax	**Move sign-extended byte to double word**
movswl	movswl %bx, %eax	**Move sign-extended word to double word**
movsbq	movsbq %bl, %rax	**Move sign-extended byte to quad word**
movswq	movswq %bx, %rax	**Move sign-extended word to quad word**
movslq	movslq %ebx, %rax	**Move sign-extended double word to quad word**
cltq	**%rax ← SignExtend(%eax)**	**Sign-extend %eax to %rax**

X86-64 Assembly Language Programming

Example : Data Movement Instructions – moving data to a register

Assembly Code

```
movq    $0x01289AB76EF34567, %rax    ;  %rax = 0x01289AB76EF34567
movb    $-1, %al                     ;  %rax = 0x01289AB76EF345FF
movw    $-1, %ax                     ;  %rax = 0x01289AB76EF3FFFF
movl    $-1, %eax                    ;  %rax = 0x00000000FFFFFFFF
movq    $-1, %rax                    ;  %rax = 0xFFFFFFFFFFFFFFFF
```

The movb instruction sets the low-order byte of %rax to FF, while the movw instruction sets the low-order 2 bytes to FFFF, with the remaining bytes unchanged.
The movl instruction sets the low-order 4 bytes to FFFFFFFF, but it also sets the higher order 4 bytes to 00000000.
Finally, the movq instruction sets the complete register to FFFFFFFFFFFFFFFF

Assembly Code

```
movq    $0x01289AB76EF34567, %rax    ;  %rax = 0x01289AB76EF34567
movb    $0xBB, %dl                   ;  %dl = 0xBB
movb    %dl, %al                     ;  %rax = 0x01289AB76EF345BB
movsbq  $dl, %rax                    ;  %rax = 0xFFFFFFFFFFFFFFBB
movzbq  $dl, %rax                    ;  %rax = 0x00000000000000BB
```

The first instruction initializes %rax to 0x01289AB76EF34567 and second instruction initializes %dl to 0xBB. The third instruction loads register %rax lower byte with value BB from register %dl.
The movsbq instruction sets the other 7 bytes to either all zeros or all ones depending on the higher order bit of the source byte. Since the lower byte value 0xBB has a higher order bit 1, the sign-extension causes the higher order bytes to each be set to FF (all 1s).
Whereas the movzbq instruction always sets all other 7 bytes to zero.

X86-64 Assembly Language Programming

INTEGER ARITHMETIC OPERATIONS

Instruction		Effect	Description
`lea`	S, D	$D \leftarrow \&S$	Load effective address
INC	D	$D \leftarrow D + 1$	Increment
DEC	D	$D \leftarrow D - 1$	Decrement
NEG	D	$D \leftarrow -D$	Negate
NOT	D	$D \leftarrow \sim D$	Complement
ADD	S, D	$D \leftarrow D + S$	Add
SUB	S, D	$D \leftarrow D - S$	Subtract
IMUL	S, D	$D \leftarrow D * S$	Multiply
XOR	S, D	$D \leftarrow D \hat{} S$	Exclusive-or
OR	S, D	$D \leftarrow D \mid S$	Or
AND	S, D	$D \leftarrow D \& S$	And
SAL	k, D	$D \leftarrow D << k$	Left shift
SHL	k, D	$D \leftarrow D << k$	Left shift (same as SAL)
SAR	k, D	$D \leftarrow D >>_A k$	Arithmetic right shift
SHR	k, D	$D \leftarrow D >>_L k$	Logical right shift

X86-64 Assembly Language Programming

Example : C and assembly codes for arithmetic function

C code

```
long arith(long x, long y, long z)
{
        long t1 = x ^ y;
        long t2 = z * 48;
        long t3 = t1 & 0x0F0F0F0F;
        long t4 = t2 – t3;
        return t4;
}
```

Assembly code

```
long arith(long x, long y, long z)
x in %rdi,   y in %rsi,   z in %rdx

arith:
        xorq  %rsi, %rdi                ; t1 = x ^ y
        leaq   (%rdx, %rdx, 2), %rax    ;%rax= 3*z
        salq   $4, %rax                 ;t2= 16 *%rax = 48*z
        andl  $0x0F0F0F0F, %edi         ; t3 = t1 & 0x0F0F0F0F
        subq %rdi, %rax                 ; return t4= %rax - %rdi
        ret
```

X86-64 Assembly Language Programming

SPECIAL ARITHMETIC INSTRUCTIONS – 32 bit

Instruction		Effect	Description
`imull`	S	R[%edx]:R[%eax] $\leftarrow S \times$ R[%eax]	Signed full multiply
`mull`	S	R[%edx]:R[%eax] $\leftarrow S \times$ R[%eax]	Unsigned full multiply
`cltd`		R[%edx]:R[%eax] $\leftarrow$ SignExtend(R[%eax])	Convert to quad word
`idivl`	S	R[%edx] $\leftarrow$ R[%edx]:R[%eax] mod S; R[%eax] $\leftarrow$ R[%edx]:R[%eax] $\div S$	Signed divide
`divl`	S	R[%edx] $\leftarrow$ R[%edx]:R[%eax] mod S; R[%eax] $\leftarrow$ R[%edx]:R[%eax] $\div S$	Unsigned divide

32-bit Multiplication: imull source e.g. imull %ebx %ebx * %eax
64-bit product will be in %edx : %eax ← %ebx * %eax

32-bit Division : idivl source e.g. idivl %ebx %edx:%eax / %ebx
For 32-bit division : 64-bit dividend is required in %edx:%eax
So sign-extend %eax to %edx to make it 64-bit using cltd instruction
Quotient will be in %eax and Remainder will be in %edx

X86-64 Assembly Language Programming

SPECIAL ARITHMETIC INSTRUCTIONS - 64 bit

Instruction	Effect	Description
imulq	S R[%rdx]:R[%rax] $\leftarrow S \times$ R[%rax]	Signed full multiply
mulq	S R[%rdx]:R[%rax] $\leftarrow S \times$ R[%rax]	Unsigned full multiply
cqo	R[%rdx]:R[%rax] $\leftarrow$ SignExtend(R[%rax])	Convert quad to Double quad word
idivq	S R[%rdx] $\leftarrow$ R[%rdx]:R[%rax] mod S	Signed divide
	R[%rax] $\leftarrow$ R[%rdx]:R[%rax] ÷ S	
divq	S R[%rdx] $\leftarrow$ R[%rdx]:R[%rax] mod S	Unsigned divide
	R[%rax] $\leftarrow$ R[%rdx]:R[%rax] ÷ S	

64-bit Multiplication: imulq source e.g. imulq %rbx operation:%rbx * %rax
128-bit product will be in %rdx : %rax ← %rbx * %rax

64-bit Division : idivq source e.g. idivq %rbx %rdx:%rax / %rbx
For 64-bit division : 128-bit dividend is required in %rdx:%rax
So sign-extend %rax to %rdx to make it 128-bit using cqo instruction
Quotient will be in %rax and Remainder will be in %rdx

X86-64 Assembly Language Programming

Example : Generation of a 128-bit product of two unsigned 64-bit numbers x and y

C code

```
#include <inttypes.h>
typedef unsigned __int128 uint128_t;
Void store_uprod(uint128_t *dest, uint64_t x, uint64_t y)
{
        *dest = x * (uint128_t) y;
}
```

Assembly code

void store_uprod(uint128_t *dest, uint64_t x, uint64_t y)
dest in %rdi, x in %rsi, y in %rdx

```
Store_uprod:
        movq  %rsi, %rax        ;copy x to multiplicand
        mulq  %rdx              ;multiply by y - 128-bit product in rdx:rax
        movq  %rax, (%rdi)      ;store product lower 8 bytes at dest
        movq  %rdx, 8(%rdi)     ;store product upper 8 bytes at dest+8
        ret
```

X86-64 Assembly Language Programming

Example : Division with x86-64 – C function & assembly code to compute Quotient and Remainder of two 64-bit signed numbers

C code

```
Void remdiv(long x, long y, long *qp, long *rp)
{
        long q = x/y;
        long  r = x%y;
        *qp = q;
        *rp = r;
}
```

Assembly code

```
void remdiv(long x, long y, long *gp, long *rp)
x in %rdi,   y in %rsi,   qp in %rdx, rp in %rcx

remdiv:
        movq  %rdx, %r8          ;copy qp address to %r8
        movq   %rdi, %rax        ;mov x to lower 8 bytes of dividend, %rax
        cqo                      ;Sign-extend to upper 8 bytes of dividend, %rdx
        idivq   %rsi             ; Divide by y , (%rdx:%rax/) /  %rsi
        movq  %rax, (%r8)        ;store quotient at qp
        movq  %rdx, (%rcx)       ;store remainder at rp
        ret
```

ARITHMETIC INSTRUCTIONS

MULTIPRECISION INTEGER ADDITION

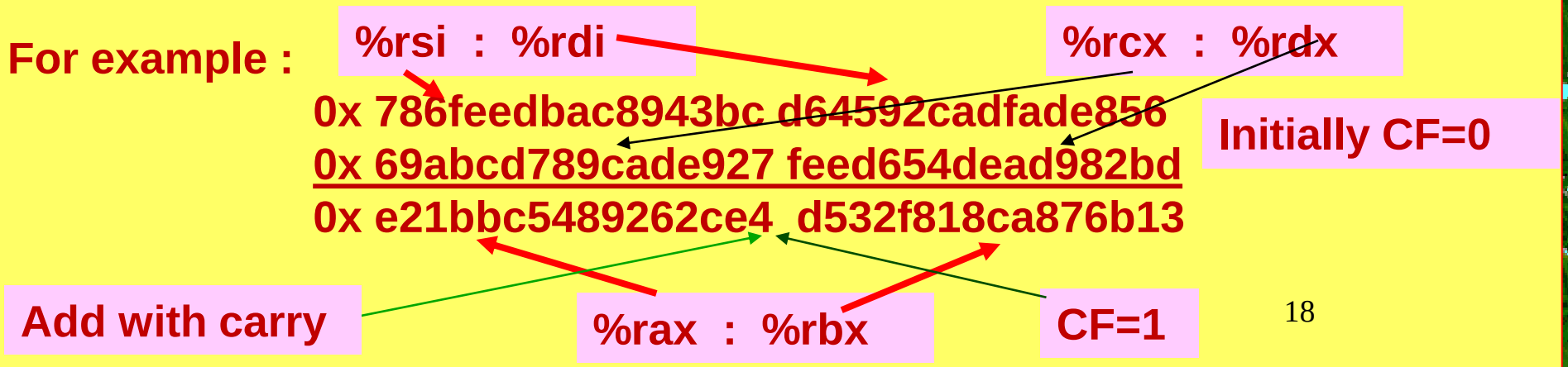
ADC - ADD WITH CARRY INSTRUCTION

ADC Src, Dst Dst ← Dst + Src + CF

Given two 128-bit integers in %rsi:%rdi and %rcx:%rdx
First 128-bit number in %rsi(higher 64-bit) and %rdi (lower 64-bit)
Second 128-bit number in %rcx(higher 64bit) and %rdx(lower 64-bit)
Perform the addition of two 128-bit numbers and store the 128-bit result in %rax(higher 64-bit) and %rbx(lower 64-bit):

```
movq  %rdi, %rbx
movq   %rsi, %rax
addq   %rdx, %rbx      #lower 64-bit addition
adcq    %rcx, %rax     # upper 64-bit addition with carry
```

For example :



ARITHMETIC INSTRUCTIONS

MULTIPRECISION INTEGER SUBTRACTION

- **SBB – SUBTRACT WITH BORROW INSTRUCTION**

SBB Src, Dst Dst← Dst – Src - CF **Borrow**

Given two 128-bit integers in %rsi:%rdi and %rcx:%rdx
First 128-bit number in %rsi(higher 64-bit) and %rdi (lower 64-bit)
Second 128-bit number in %rcx(higher 64bit) and %rdx(lower 64-bit)
Perform the subtraction of two 128-bit numbers and store the 128-bit result in %rax(higher 64-bit) and %rbx(lower 64-bit):

```
movq  %rdi, %rbx
movq   %rsi, %rax
subq   %rdx, %rbx      #lower 64-bit subtraction
sbbq   %rcx, %rax      # upper 64-bit subtraction with borrow
```

For example :

%rsi : %rdi **%rcx : %rdx**

0x 786feedbac8943bc d64592cadfade856
0x 69abcd789cade927 feed654dead982bd
0x 0fc421630fdb5a94 d6582d7cf5d46599

Initially CF=0

Subtract with borrow **%rax : %rbx** **CF=1 Borrow**

SOC 2040 SYSTEM PROGRAMMING

Reading Assignments

Chapter 3 of Text Book

- **Computer Systems : A Programmer's Perspective**
  - **Randal E. Bryant and David R. O'Hallaron 3rd Edition, Global Edition Prentice Hall 2016 ISBN 10:1-292-10176-8**

SOC 2040
SYSTEM PROGRAMMING

END OF WEEK6
LECTURE1

MACHINE-LEVEL PROGRAMMING I

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
SPRING SEMESTER 2024

SOC 2040
SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG